



AI STUDENT COLLECTIVE MADRID

Propuesta de proyecto: Skilache

Héctor Molina

hector.molina@alumnos.uc3m.es

@Toriomg

Madrid, 24 Septiembre 2026

1. Título y resumen (TL;DR¹)

Skilache sería un motor de búsqueda que, a partir del catálogo de asignaturas y grupos de una titulación de la UC3M, encontrara el **horario óptimo** para un alumno según sus preferencias: menos horas muertas, menos días con clase, franjas horarias u horas concretas a evitar o priorizar, mañanas o tardes, mismo grupo entre asignaturas... entre otros. Por debajo, no deja de ser resolver un CSP (qué combinaciones de grupos no se pisan) y luego un problema de búsqueda sobre esas combinaciones (cuál puntúa mejor).

2. Introducción

NOTA: Se introducen notas al pie de página de conceptos básicos que puedan no ser familiares para un estudiante de 1º de carrera.

2.1. Motivación

Hacer los horarios de la uni sin que se solapen, se te acumulen los días, etc. es profundamente tedioso.

2.2. Objetivo

Implementar un optimizador de horarios universitario para los alumnos de la UC3M que, a partir del catálogo de asignaturas y grupos disponibles, genere el **horario óptimo** según las preferencias del alumno. Luego, se podría presentar a la universidad.

2.3. Nombre

Había pensado en hacer un juego de palabras entre "Scheduler" y "Esquilache" (Ministro de Hacienda en la época de Carlos III): "Schilache" la primera "ch" no la van a pronunciar como una "q", "Esquilacheduler" no me convence porque es muy largo, **"Skilache"** es la que más me gusta, siendo un plus si ponemos de logo/mascota del proyecto el ski (). Demasiada identidad le estoy dando ya jajaja.

→ **Abierto a propuestas.**

3. Desarrollo

3.1. Scope

El scope inicial debería estar enfocado a resolver el problema principal de garantizar que **no haya solapamientos y se pueda minimizar el número de horas muertas** entre clases.

¿Qué entraría dentro del **MVP**²?

- Gestión de un único alumno.
- Un conjunto reducido de criterios de optimización: horas muertas, días libres y franja horaria preferida.
- Catálogo de asignaturas introducido manualmente o vía ficheros CSV, JSON, etc. (sin scraping³ automático de fuentes oficiales por ahora).
- Interfaz mínima, GUI⁴ simple para introducir preferencias y ver el resultado.
- Validación sobre una única titulación como prueba de concepto.

Quedan fuera de esta primera versión, como líneas futuras:

- Integración con sistemas de la UC3M.
- Soporte generalizado a todas las titulaciones.
- Crear una caché server-side para consultas repetidas.

¹TL;DR: Too long; didn't read

²MVP: Minimum viable product

³Scraping: Técnica utilizada mediante programas de software para extraer información de sitios web.

⁴GUI: «Graphic User Interface»

3.2. Funcionalidades

- Carga del catálogo de asignaturas y grupos.
- Selección por parte del alumno de las asignaturas a cursar.
- Definición de preferencias: franja horaria preferida, días libres deseados, etc.
- Motor de búsqueda que descarte combinaciones con solapamientos y calcule las horas muertas de cada horario válido.
- Generación del horario óptimo (o de los N mejores) según los criterios anteriores.
- Visualización del resultado (tabla u horario en formato calendario) por CLI o formulario simple.

3.3. Enfoque

3.3.1. Definición del problema

Para los estudiantes de 3º de Informática y MatComp, y 2º de IA; en la asignatura de Heurística y Optimización (dónde toca hacer malabares para aprobar) se dan todos los fundamentos teóricos⁵ para el funcionamiento del motor de búsqueda. En el **Apéndice A**, se encuentran explicados.

El problema de generar un horario se puede modelar como un **problema de satisfacción de restricciones** (CSP⁶):

- **Variables:** cada asignatura que el alumno quiere cursar.
- **Dominio:** los grupos disponibles para esa asignatura (cada grupo lleva asociado un horario concreto).
- **Restricciones:** las asignaciones deben ser consistentes entre sí, siendo la principal que no haya solapamientos horarios entre los grupos elegidos.

Resolver el CSP da el conjunto de horarios **válidos**, sin solapamientos, pero no resuelve cuál es el **mejor**. Para ello, cada horario válido se evalúa con una función objetivo que traduce las preferencias del alumno (horas muertas, días libres, franja horaria...) en una puntuación numérica. Encontrar el horario que maximiza/minimiza esa puntuación es entonces un **problema de búsqueda**⁷ sobre el espacio de soluciones del CSP.

Estaría bien consultar proyectos o soluciones similares.

3.3.2. Sistema

Una propuesta de arquitectura para un sistema completo sería la siguiente:

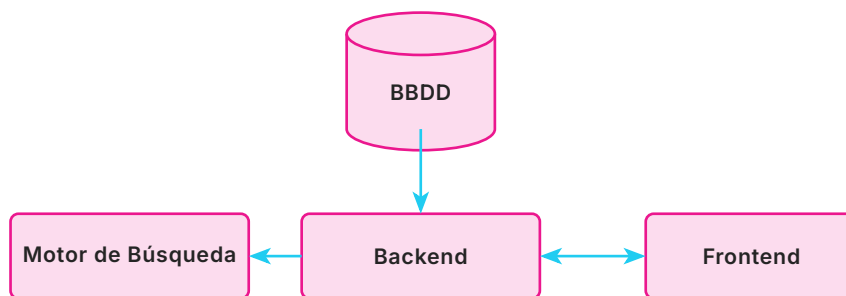


Figura 1: Diseño del sistema

⁵ Aunque no sea la IA "trendy" (LLMs, Transformers, ML, Deep Learning, ...) de la que se habla ahora mismo, la búsqueda y los CSP son la IA "de toda la vida": llevan décadas siendo la forma estándar de resolver problemas con soluciones exactas y verificables, justo lo que se necesita aquí, y no un modelo entrenado con datos.

⁶CSP: «Constraint Satisfaction Problem». Un problema donde hay que asignar valores a un conjunto de variables cumpliendo ciertas restricciones.

⁷ Problema de búsqueda: dado un espacio de posibles soluciones (aquí, los horarios válidos), encontrar la que mejor cumple un criterio dado, explorando dicho espacio de forma sistemática en lugar de probar todas las combinaciones una por una.

3.3.2.1. Motor de búsqueda

Lo más importante es que el motor sea rápido, por lo que se descartaría cualquier lenguaje interpretado (Python). Unas opciones perfectas sería usar C++ o Rust. Dado que, C++ es un lenguaje que requiere mucha experiencia para lograr algo “que no sea malo” y Rust tiene una buena barrera de entrada, creo que la mejor opción sería Go⁸ al ser un lenguaje compilado, valga la redundancia.

Se encargaría de buscar los posibles horarios (estados) resolviendo el problema CSP, resolviendo después el problema de búsqueda adyacente para encontrar el horario “óptimo”.

3.3.2.2. Backend

Se encarga de comunicarse con la BBDD y el motor para compartir los datos al Frontend. Habría que dockerizar⁹ el backend, y programarlo en Python. Para el protocolo de comunicación con el frontend no tiene que ser complejo. Se podría usar RPC (xmlrpc, gRPC, ...) o una API REST (FastAPI, Flask, ...)

3.3.2.3. Frontend

Lo idóneo sería una interfaz web, usando **tecnología web nativa (HTML, CSS y JavaScript)**, qué es la que requiere saber menos de antes. Aunque esta sección se puede delegar a la IA.

3.3.2.4. Base de datos

Para la BBDD, almacenaría inicialmente en un .json todos los horarios de una titulación. Al escalar a todas las titulaciones habría que ver cuánto engrosarían los datos a tratar, y entonces habría que decidir si ir a lo simple con un SQLite, o ir a BBDD más grandes como MongoDB.

4. Plan de trabajo

A decidir

5. Equipo

A decidir

6. Recursos necesarios

A decidir

⁸ Go es un lenguaje desarrollado por Google en 2007 con la finalidad de resolver problemas de lentitud, complejidad y baja productividad que tenían al desarrollar software a gran escala. Destaca por la simpleza de su diseño y su facilidad de aprenderlo. Es un lenguaje ciertamente demandado para cloud y sistemas. Yo tampoco lo sé, así que es una buena oportunidad para aprenderlo.

⁹ Docker: aplicación para empaquetar una aplicación junto con todas sus dependencias, códigos y bibliotecas en un paquete ligero llamado contenedor.

A. Fundamentos: CSP y problemas de búsqueda

Esto lo ha redactado la IA y posteriormente lo he revisado.

Este anexo desarrolla un poco más los dos conceptos de Heurística y Optimización sobre los que se apoya Skilache, que en [Sección 3.3](#) solo se mencionan por encima. Es la chuleta a la que recurrir a la hora de diseñar el motor de verdad.

Problema de satisfacción de restricciones (CSP)

Ya lo dijimos en [Sección 3.3](#): generar un horario es, en el fondo, un CSP. La idea es sencilla, aunque suene rimbombante: formalmente un CSP se define como una terna (X, D, C) , donde $X = \{x_1, \dots, x_n\}$ son las **variables** (las asignaturas), $D = \{d_1, \dots, d_n\}$ son los **dominios** (los grupos en los que se puede matricular cada una) y C es el conjunto de **restricciones** que dicen qué combinaciones de valores están permitidas (que los horarios elegidos no se pisen entre sí). Una **asignación** es una función que le da un valor de su dominio a cada variable; es **completa** si cubre todas las variables y **consistente** si no rompe ninguna restricción. Resolver el CSP es, ni más ni menos, encontrar una asignación completa y consistente.

El problema es que, si lo intentamos por fuerza bruta probando todas las combinaciones de grupos una por una, la cosa explota rápido: el número de combinaciones es el producto del número de grupos de cada asignatura, así que con un puñado de asignaturas ya tenemos miles (o millones) de horarios que probar¹⁰. Por suerte hay formas mucho más listas de resolverlo:

- **Backtracking**¹¹: en vez de generar horarios completos y luego comprobar si valen, se va asignando un grupo a cada asignatura de una en una, y en cuanto dos grupos ya asignados se pisan se corta esa rama ahí mismo, sin perder tiempo completándola.
- **Forward checking**: cada vez que se fija un grupo para una asignatura, se eliminan de golpe de las asignaturas pendientes los grupos que ya sabemos que chocarían con ese. Así se detectan los callejones sin salida antes, no al final.
- **Arc consistency (AC-3)**: una versión aún más agresiva de lo anterior, que propaga estas eliminaciones entre pares de asignaturas antes incluso de empezar a asignar, para llegar al backtracking con los dominios ya «podados».
- Y para no elegir el orden de asignaturas/grupos a lo tonto, unas heurísticas de toda la vida: asignar antes la asignatura que tenga **menos grupos disponibles** (MRV, porque si va a fallar, mejor que falle pronto), desempatar por la que más restricciones tenga con el resto (degree heuristic), y al elegir grupo, probar antes el que menos cierre puertas a las asignaturas que quedan (least constraining value).

Ojo: resolver el CSP nos da **todos** los horarios válidos, pero al CSP le da exactamente igual cuál de ellos es mejor para el alumno — para el solver, todos son igual de «correctos». Ahí es donde entra el segundo concepto.

Problema de búsqueda

Un problema de búsqueda es, básicamente, «tengo un montón de estados posibles conectados entre sí, quiero llegar a uno que cumpla cierta condición (o que sea el mejor posible), y necesito una forma sistemática de explorarlos sin mirarlos todos uno por uno». Formalmente se define como una tupla $(s_0, \text{Acciones}, \text{Transición}, \text{Test-objetivo}, \text{Coste})$. Para nosotros:

- Un **estado inicial** (para nosotros: no hay ninguna asignatura asignada todavía).
- Unas **acciones** que llevan de un estado a otro (asignar un grupo concreto a la siguiente asignatura).
- Un **test de objetivo** que dice si ya hemos llegado (todas las asignaturas tienen grupo y no hay solapamientos).

¹⁰Esto es justo lo que hace que un CSP sea, en general, NP-completo: no se conoce ninguna forma de resolverlo siempre rápido, y el coste puede crecer exponencialmente con el número de variables.

¹¹Backtracking: ir asignando variables una a una y, en cuanto la asignación parcial deja de tener sentido (viola una restricción), deshacer esa última decisión y probar otra en vez de seguir construyendo sobre un error.

- Una forma de puntuar cada solución (horas muertas, días libres...) para poder decir cuál es mejor que cuál.

Todos esos estados y las conexiones entre ellos forman el **espacio de búsqueda**, que se suele imaginar como un árbol que se va recorriendo poco a poco en vez de generarlo entero de golpe.

Aquí hay dos familias de algoritmos:

- Las que **no usan ninguna pista** sobre el problema (BFS, DFS, búsqueda de coste uniforme): funcionan, encuentran solución si existe, pero van un poco a ciegas.
- Las que sí usan una **heurística**¹², que estima «qué tan bien pinta» un estado antes de terminar de explorarlo. La estrella aquí es A^* ¹³, que combina lo ya recorrido con la estimación de lo que falta para no perder tiempo en ramas que claramente no van a llegar a nada bueno.

Como en Skilache no nos vale **cualquier** horario válido sino **el mejor** según las preferencias del alumno, esto se parece bastante a un problema de optimización, así que también merece la pena tener en el radar algo de **búsqueda local**: partir de una solución y moverse a vecinos que la mejoren (hill climbing), aceptando de vez en cuando un paso peor para no quedarse atascado en un óptimo que solo es bueno «en su barrio» (simulated annealing), o llevar varias soluciones candidatas en paralelo en vez de una sola (beam search).

Trasladado a Skilache: el **estado** es una asignación parcial de grupos, el **objetivo** es una asignación completa y sin solapamientos que además puntúe lo mejor posible, y la heurística natural es algo como «cuántas horas muertas lleva ya acumuladas esta asignación a medias» — si ya va peor que la mejor solución que tenemos hasta ahora, ni se sigue por ahí (la poda que se comenta en [Sección 3.3](#)).

Cómo se juntan los dos

Lo bueno es que no hace falta resolver el CSP entero primero y luego, aparte, buscar el mejor horario entre todos los válidos: eso obligaría a generarlos todos, que es justo lo que queremos evitar. Lo normal es hacerlo todo en el mismo recorrido tipo backtracking:

1. Se construye la asignación asignatura a asignatura, como en un CSP.
2. En cada paso se aplica forward checking / AC-3 para descartar cuanto antes las ramas que ya no pueden llevar a nada válido.
3. A la vez, se lleva la cuenta de la puntuación parcial (horas muertas, etc.) y se compara con la mejor solución completa encontrada hasta el momento: si esta rama ya no puede superarla, se corta ahí sin seguir explorando (el mismo espíritu que el $g(n)$ de A^*).

Así el CSP pone la estructura del problema (qué combinaciones son válidas) y la búsqueda pone el criterio para decidir por dónde merece la pena seguir mirando y cuándo dejar de perder el tiempo en una rama que ya no va a ganar.

¹²Heurística: una estimación rápida (no necesariamente exacta) de lo buena que puede llegar a ser una solución a partir de un estado a medio construir, para decidir por dónde merece la pena seguir explorando.

¹³ A^* : explora primero los estados con mejor $g(n) + h(n)$, donde $g(n)$ es lo que ya ha costado llegar hasta ahí y $h(n)$ es la estimación de lo que falta. Si la estimación nunca es demasiado optimista, A^* garantiza encontrar la solución óptima sin tener que explorarlo todo.